

Java Básico — Explicação Detalhada

O que é Java?

Java é uma linguagem de programação criada pela Oracle Corporation. Ela é orientada a objetos, multiplataforma e amplamente utilizada no desenvolvimento de sistemas empresariais, aplicações web, APIs, aplicativos Android e sistemas embarcados.

O lema do Java é:

"Write Once, Run Anywhere" (Escreva uma vez, execute em qualquer lugar)

Isso significa que um programa Java pode ser executado em diferentes sistemas operacionais sem precisar ser reescrito.

Como o Java funciona?

Quando você escreve um código Java, ele não é transformado diretamente em código de máquina.

O processo é:

1. Você escreve o código (`.java`)
2. O compilador (`javac`) transforma o código em Bytecode (`.class`)
3. A JVM (Java Virtual Machine) executa esse Bytecode

Fluxo:

```
Código Java (.java)
  ↓
Compilador (javac)
  ↓
Bytecode (.class)
  ↓
JVM
  ↓
Execução
```

Estrutura de um programa Java

Exemplo:

```
public class Main {  
  
    public static void main(String[] args) {  
  
        System.out.println("Olá Mundo!");  
  
    }  
  
}
```

Explicação

public class Main

Define uma classe chamada **Main**.

```
public class Main
```

Toda aplicação Java precisa estar dentro de uma classe.

Método main

```
public static void main(String[] args)
```

É o ponto de entrada do programa.

Quando o Java executa um programa, ele procura esse método primeiro.

System.out.println()

```
System.out.println("Olá Mundo!");
```

Serve para exibir informações no console.

Resultado:

Olá Mundo!

Variáveis

Variáveis armazenam dados na memória.

Exemplo:

```
String nome = "Maria";  
int idade = 20;  
double altura = 1.70;  
boolean estudante = true;
```

Tipos de Dados Primitivos

int

Números inteiros.

```
int idade = 25;
```

double

Números decimais.

```
double salario = 2500.50;
```

char

Um único caractere.

```
char letra = 'A';
```

boolean

Valores verdadeiro ou falso.

```
boolean ativo = true;
```

byte

Inteiros pequenos.

```
byte numero = 100;
```

short

Inteiros médios.

```
short valor = 30000;
```

long

Inteiros grandes.

```
long populacao = 8000000000L;
```

float

Decimais de menor precisão.

```
float preco = 19.99f;
```

String

String não é um tipo primitivo.

É uma classe usada para armazenar textos.

```
String nome = "João";
```

Exemplo:

```
String nome = "Carlos";
```

```
System.out.println(nome);
```

Resultado:

Carlos

Operadores

Soma

```
int resultado = 10 + 5;
```

Resultado:

15

Subtração

```
int resultado = 10 - 5;
```

Multiplicação

```
int resultado = 10 * 5;
```

Divisão

```
int resultado = 10 / 5;
```

Resto da divisão

```
int resultado = 10 % 3;
```

Resultado:

1

Entrada de Dados

Para receber dados do usuário usamos Scanner.

```
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.print("Digite seu nome: ");

        String nome = scanner.nextLine();

        System.out.println("Olá " + nome);

    }

}
```

Estruturas Condicionais

if

```
int idade = 18;

if (idade >= 18) {
    System.out.println("Maior de idade");
}
```

if e else

```
if (idade >= 18) {
```

```
        System.out.println("Maior de idade");
    } else {
        System.out.println("Menor de idade");
    }
}
```

if, else if e else

```
int nota = 8;
```

```
if (nota >= 9) {
    System.out.println("Excelente");
} else if (nota >= 7) {
    System.out.println("Aprovado");
} else {
    System.out.println("Reprovado");
}
```

Switch

```
int opcao = 2;
```

```
switch (opcao) {
    case 1:
        System.out.println("Cadastrar");
        break;

    case 2:
        System.out.println("Editar");
        break;

    default:
        System.out.println("Opção inválida");
}
```

Estruturas de Repetição

for

```
for (int i = 1; i <= 5; i++) {
```

```
    System.out.println(i);  
}
```

Saída:

```
1  
2  
3  
4  
5
```

while

```
int i = 1;  
  
while (i <= 5) {  
    System.out.println(i);  
    i++;  
}
```

do while

```
int i = 1;  
  
do {  
    System.out.println(i);  
    i++;  
} while (i <= 5);
```

Arrays

Arrays armazenam vários valores do mesmo tipo.

```
int[] numeros = {10, 20, 30, 40};
```

Acessando posições:

```
System.out.println(numeros[0]);
```


Resultado:

10

Métodos

Métodos servem para reutilizar código.

Exemplo:

```
public static void saudacao() {  
    System.out.println("Olá!");  
}
```

Chamando:

```
saudacao();
```

Método com retorno:

```
public static int somar(int a, int b) {  
    return a + b;  
}
```

Uso:

```
int resultado = somar(10, 20);
```

```
System.out.println(resultado);
```

Programação Orientada a Objetos (POO)

Java é uma linguagem orientada a objetos.

Os principais conceitos são:

Classe

Molde para criar objetos.

```
public class Pessoa {  
}
```

Objeto

Instância de uma classe.

```
Pessoa p = new Pessoa();
```

Atributos

Características do objeto.

```
public class Pessoa {  
  
    String nome;  
    int idade;  
  
}
```

Métodos

Comportamentos do objeto.

```
public class Pessoa {  
  
    String nome;  
  
    void falar() {  
        System.out.println("Olá!");  
    }  
  
}
```

Construtor

Executado quando o objeto é criado.

```
public class Pessoa {  
  
    String nome;  
  
    public Pessoa(String nome) {  
        this.nome = nome;  
    }  
  
}
```

Uso:

```
Pessoa p = new Pessoa("Maria");
```

Encapsulamento

Protege os dados da classe.

```
private String nome;
```

Uso de getters e setters:

```
public String getNome() {  
    return nome;  
}  
  
public void setNome(String nome) {  
    this.nome = nome;  
}
```

Herança

Permite reutilizar código.

```
public class Animal {  
  
    public void emitirSom() {  
        System.out.println("Som");  
    }  
  
}
```

```
}
```

```
public class Cachorro extends Animal {
```

```
}
```

Polimorfismo

Permite sobrescrever métodos.

```
public class Animal {
```

```
    public void emitirSom() {  
        System.out.println("Som genérico");  
    }  
}
```

```
}
```

```
public class Cachorro extends Animal {
```

```
    @Override  
    public void emitirSom() {  
        System.out.println("Au Au");  
    }  
}
```

```
}
```

Coleções

ArrayList

Lista dinâmica.

```
import java.util.ArrayList;
```

```
ArrayList<String> nomes = new ArrayList<>();
```

```
nomes.add("João");  
nomes.add("Maria");  
nomes.add("Carlos");
```

Percorrendo:

```
for (String nome : nomes) {  
    System.out.println(nome);  
}
```

Tratamento de Exceções

Evita que o programa pare devido a erros.

```
try {  
  
    int resultado = 10 / 0;  
  
} catch (Exception e) {  
  
    System.out.println("Erro encontrado");  
  
}
```

Resumo

Os fundamentos mais importantes para aprender Java são:

1. Sintaxe básica
2. Variáveis e tipos de dados
3. Operadores
4. Entrada e saída de dados
5. Estruturas condicionais
6. Estruturas de repetição
7. Arrays
8. Métodos
9. Classes e Objetos
10. Encapsulamento
11. Herança
12. Polimorfismo
13. Coleções
14. Tratamento de exceções

Depois de dominar esses tópicos, o próximo passo normalmente é estudar Collections Framework, Streams, Lambdas, JDBC, APIs REST com Spring Boot e bancos de dados relacionais.